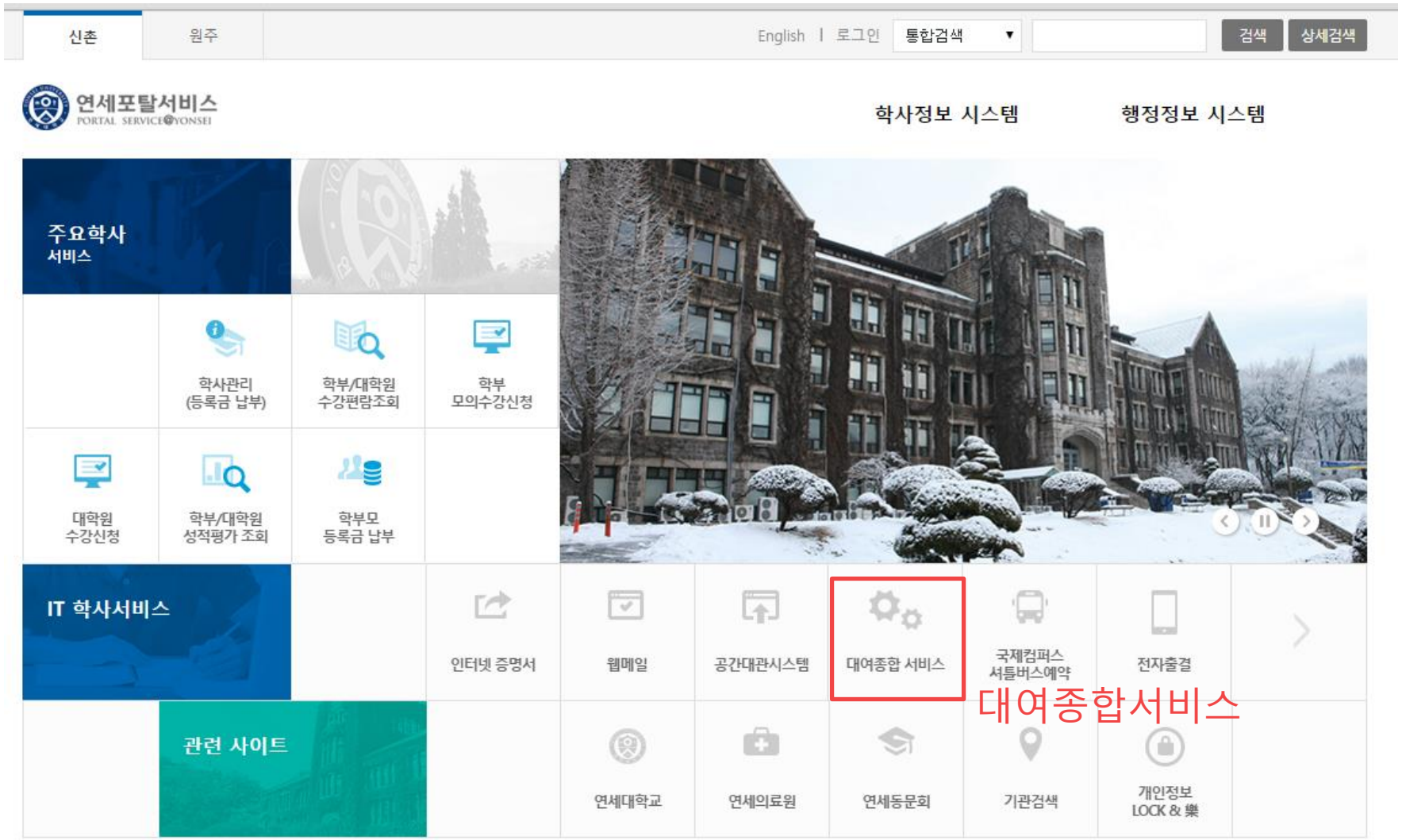
Astrodynamics 2

Introduction to MATLAB
Hohmann transfer trajectory through MATLAB

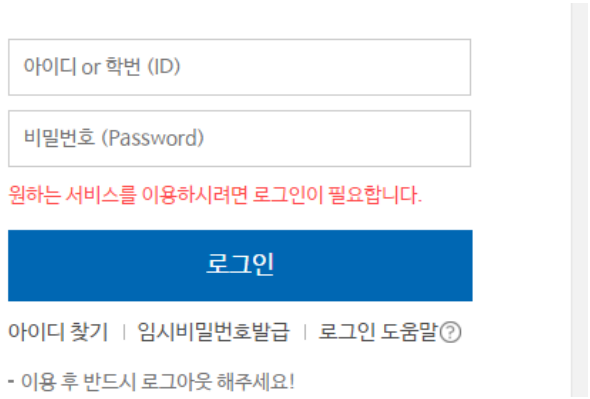
2020.10.08

김나은

MATLAB Installation



MATLAB Installation

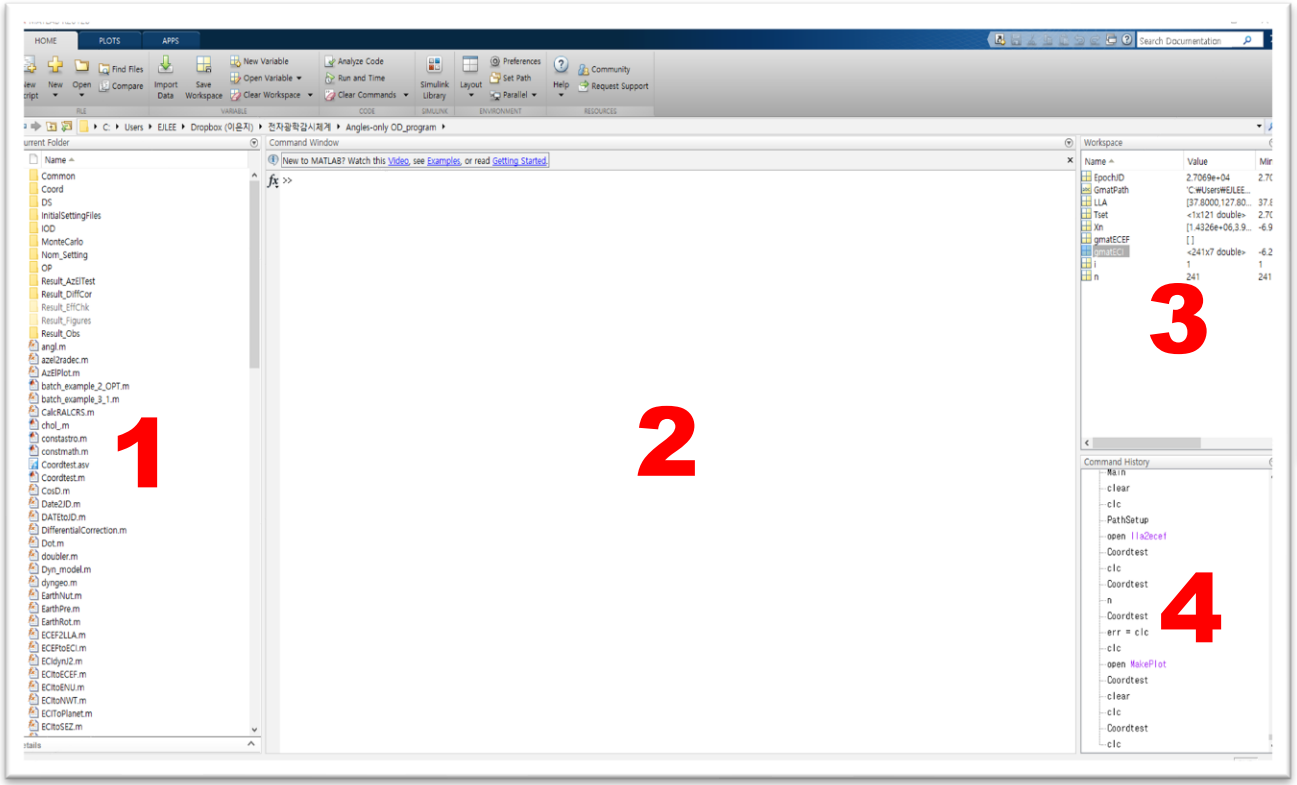


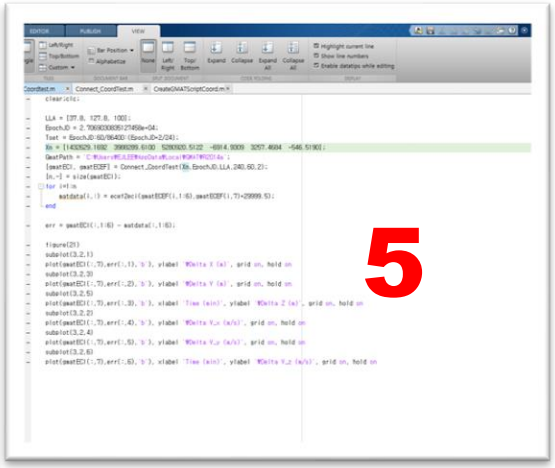
소프트웨어 신청 → 로그인→ 소프트웨어→MATLAB R2012b

Adobe Acrobat Document

Introduction to MATLAB

- MATLAB?
- MATLAB is a engineering software for numerical analysis and programming developed by MathWorks
- Slower than C/C++, but has user friendly interfaces





1. **Current directory**
2. **Command window**
3. **Workspace**
4. **Command history**
5. **Editor**

Introduction to MATLAB

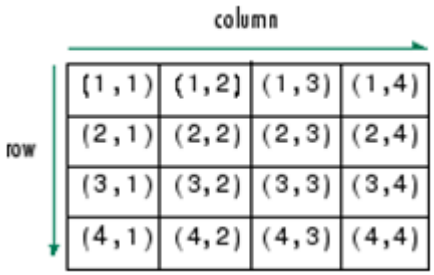
- Basic operations: Variables
- Example:

```
>> x=1;
>> y=x+1;
```

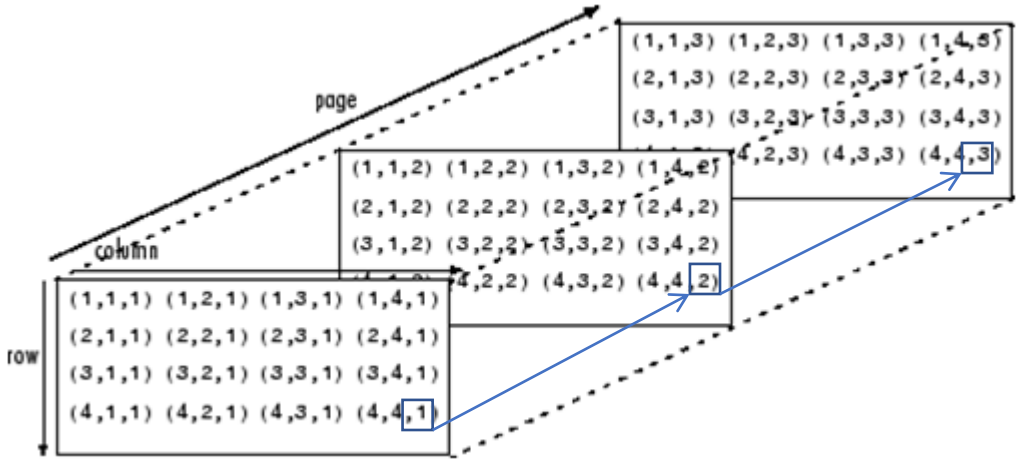
- Results
  - First, x will be added into the workspace with a value of 1
  - With the second command, y will appear with a value 2
  - Here, ';' hides the operation result in the command window

Introduction to MATLAB

- Basic operations: array
- A collection of data values organized into rows and columns, and known by a single name.



2차원 배열: i와 j 두 개의 index를 사용해서 데이터를 불러내거나 저장한다.



3차원 배열: 각각의 2차원 배열을 page로 간주하여 그림과 같이 데이터를 저장한다.

Introduction to MATLAB

- Basic operations

$$a = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix}$$

3x2 matrix → 6 elements

b=[1 2 3 4]

1x4 array → 4 elements, row vector

$$c = \begin{bmatrix} 1 \\ 3 \\ 5 \end{bmatrix}$$

3x1 array → 3 elements, column vector

a(2,1)=3 **b(3)=3** **c(2)=3**

Row # Column #

Introduction to MATLAB

- Basic operations
  - Matrix

```
>> A=[1,2,3;4,5,6;7,8,9];
```

or

```
>> A=[1 2 3
      4 5 6
      7 8 9];
```

  - Operations

```
>> transpose(A) (or A')
>> inv(A) (or A^-1)
>> A*B
>> A+B
```

```
>> A=eye(3,3)

A =

     1     0     0
     0     1     0
     0     0     1

>> B=[1;1]

B =

     1
     1

>> A*B
??? Error using ==> mtimes
Inner matrix dimensions must agree.

>> A+B
??? Error using ==> plus
Matrix dimensions must agree.
```

※ note: check the dimension of the matrix before operation

Introduction to MATLAB

- Basic operations

```
>> A=rand(3,3)

A =

    0.8147    0.9134    0.2785
    0.9058    0.6324    0.5469
    0.1270    0.0975    0.9575

>> A(2)

ans =

    0.9058

>> A(4)

ans =

    0.9134
```

```
>> A=rand(3,3)

A =

    0.9649    0.9572    0.1419
    0.1576    0.4854    0.4218
    0.9706    0.8003    0.9157

>> A(2,2)

ans =

    0.4854
```

```
>> A=rand(3,3)

A =

    0.7922    0.0357    0.6787
    0.9595    0.8491    0.7577
    0.6557    0.9340    0.7431

>> A(2,:)

ans =

    0.9595    0.8491    0.7577
```

Introduction to MATLAB

- Basic operations
  - Colon operator

```
>> x=1:2:13

x =

    1     3     5     7     9    11    13
```

✓ Returns a arithmetical progression from 1 to 13 with a interval of 2

  - Element operation

```
>> A=[1,2;3,4];B=[3,4;1,2];
>> A*B

ans =

    5     8
   13    20
```

```
>> A.*B

ans =

    3     8
    3     8
```

Introduction to MATLAB

- Useful commands

>> size(A)

: Returns the dimension of the matrix

>> length(A)

: Returns the length of the vector(matrix)

>> clear

: clear all variables in the workspace

>> clc

: clear command window

```
>> A=eye(3)

A =

     1     0     0
     0     1     0
     0     0     1

>> size(A)

ans =

     3     3
```

```
>> A=rand(4,3)

A =

    0.3922    0.0318    0.8235
    0.6555    0.2769    0.6948
    0.1712    0.0462    0.3171
    0.7060    0.0971    0.9502

>> length(A)

ans =

     4
```

Introduction to MATLAB

- Useful commands

>> sqrt(x)

: Returns the square root of the vector

>> abs(x)

: Returns the absolute value

>> zeros(n,m)

: Returns n by m matrix with all elements are zero

>> ones(n,m)

: Returns n by m matrix with all elements are one

>> eye(n)

: Returns n square identity matrix

Introduction to MATLAB

- Useful commands

>> load ‘filename.mat’

: load filename.mat to workspace in order to use it in code

>> importdata(‘filename.txt’)

: import file to workspace in order to use it in code

>> disp ‘Hello World’

: display “Hello World” to Command Window

>> diag(A)

: pick diagonal term of the matrix A or make diagonal matrix if A is array.

>> struct.xx

: Make structure named struct which has xx inside

Introduction to MATLAB

- Operators
  - Relational
    : If true, returns 1 or else 0

```
>> A=2;

>> A>1

ans =

    1

>> A>2

ans =

    0

>> A>=2

ans =

    1
```

```
>> A==2

ans =

    1

>> A~=2

ans =

    0
```

- Logical
  : &(and), |(or), ~(not)

```
>> 1&1

ans =

    1
```

```
>> 1&0

ans =

    0
```

```
>> 1|0

ans =

    1
```

Introduction to MATLAB

- Pre-allocation

Note: while the variable is extended in the loop, the process can be much reduced with pre-allocation.

C:₩Documents and Settings₩youngkwang₩My Documents₩MATLAB

```
T=12000;
h=6.63e-34;
c=3e8;
k=1.38e-23;

for j=1:1:2000
lambda(j)=j*10^-9;
B(j)=2.*h.*c.^2./lambda(j).^5./(exp(h.*c./k./T./lambda(j))-1);

end
plot(lambda,B)
```

C:₩Documents and Settings₩youngkwang₩My Documents₩MATLAB₩

```
T=12000;
h=6.63e-34;
c=3e8;
k=1.38e-23;

lambda=zeros(1,2000);
B=zeros(1,2000);

for j=1:1:2000
lambda(j)=j*10^-9;
B(j)=2.*h.*c.^2./lambda(j).^5./(exp(h.*c./k./T./lambda(j))-1);

end
plot(lambda,B)
```

Pre-allocating

Vectors and Matrices

- Character

>> s='hello'

length_s=size(s)

```
length_s =

     1     5
```

- Save row vector (1x5 array)

s(2:4)

```
>> s(2:4)

ans =

ell
```

- Character array (char data) is also accessed by index (double data)

Vectors and Matrices

- Conversion between double and char
  - ➢ function double : char data → ascii code

```
>> double(s)

ans =

   104   101   108   108   111
```

  - ➢ function char : double data → char data

```
>> char(65:90)

ans =

ABCDEFGHIJKLMNOPQRSTUVWXYZ
```

TABLE 3
ASCII CHARACTER CODES (DECIMAL)

0	Ctrl-@	32	Space	64	@	96	‘
1	Ctrl-A	33	!	65	A	97	a
2	Ctrl-B	34	"	66	B	98	b
3	Ctrl-C	35	#	67	C	99	c
4	Ctrl-D	36	$	68	D	100	d
5	Ctrl-E	37	%	69	E	101	e
6	Ctrl-F	38	&	70	F	102	f
7	Ctrl-G	39	’	71	G	103	g
8	Backspace	40	(	72	H	104	h
9	Tab	41	)	73	I	105	i
10	Ctrl-J	42	*	74	J	106	j
11	Ctrl-K	43	+	75	K	107	k
12	Ctrl-L	44	,	76	L	108	l
13	Return	45	-	77	M	109	m
14	Ctrl-N	46	.	78	N	110	n
15	Ctrl-O	47	/	79	O	111	o
16	Ctrl-P	48	0	80	P	112	p
17	Ctrl-Q	49	1	81	Q	113	q
18	Ctrl-R	50	2	82	R	114	r
19	Ctrl-S	51	3	83	S	115	s
20	Ctrl-T	52	4	84	T	116	t
21	Ctrl-U	53	5	85	U	117	u
22	Ctrl-V	54	6	86	V	118	v
23	Ctrl-W	55	7	87	W	119	w
24	Ctrl-X	56	8	88	X	120	x
25	Ctrl-Y	57	9	89	Y	121	y
26	Ctrl-Z	58	:	90	Z	122	z
27	Escape	59	;	91	[	123	{
28	Ctrl-\	60	<	92	\	124	|
29	Ctrl-]	61	=	93	]	125	}
30	Ctrl-^	62	>	94	^	126	~
31	Ctrl-_	63	?	95	_	127	Delete

Vectors and Matrices

- Convergence of array
  - Bracket
    - ✓ spacing of the words is not permitted
    - ✓ an error may occur when the array size is mismatched

```
>> x='seoul';
>> y='korea';
>> z=[x y]

z =

seoulkorea
```

  - Using char
    - ✓ the longest array makes size of matrix
    - ✓ abc = 3x7 array

```
>> a='apple';
>> b='orange';
>> c='bananas';
>> abc=char(a,b,c)

abc =

apple
orange
bananas
```

MATLAB Basics

- The num2str() and int2str() functions

```
>> d = [ num2str(16) '-Feb-' num2str(2004) ];
>> disp(d)
16-Feb-2004

>> x = 23.11;
>> disp( [ 'answer = ' num2str(x) ] )
answer = 23.11

>> disp( [ 'answer = ' int2str(x) ] )
answer = 23
```

MATLAB Basics

- The fprintf(format, data) functions

```
>> fprintf( 'Result is %d', 3 )
Result is 3
>> fprintf( 'Area of a circle with radius %d is %f', 3, pi*3^2 )
Area of a circle with radius 3 is 28.274334
>> x = 5;
>> fprintf( 'x = %3d', x )
x = 5
>> x = pi;
>> fprintf( 'x = %0.2f', x )
x = 3.14
>> fprintf( 'x = %6.2f', x )
x = 3.14
>> fprintf( 'x = %d\ny = %d\n', 3, 13 )
x = 3
y = 13
```

%d : integer
%f : floating point format
%e : exponential format
%g : either floating point or exponential format, whichever is shorter
\n : new line character
\t : tab character

MATLAB Basics

- The format options
  - Changing the data format

```
>> value = 12.345678901234567;
```

format short → 12.3457

format long → 12.34567890123457

format short e → 1.2346e+001

format long e → 1.234567890123457e+001

format short g → 12.346

format long g → 12.3456789012346

format rat → 1000/81

Function

Symbolic math

- 해석적인 접근을 위해 변수를 문자로 취급하여 계산
- symbolic math toolbox가 설치되어 있어야 사용 가능

- Example:

```
>>syms x y z
>>y=z/x
y =
z/x
```

```
>>syms y x n
>>y=x^n
>>diff(y,x)
ans =
x^n*n/x
```

```
>> syms x y
>> y=x^2;
>> int(y,x)
 ans =
x^3/3
```

Function handle

- Function handle이란?
  - 각각의 함수가 가지는 고유한 값
  - MATLAB에서 함수들의 이름은 같을 수 있지만 각각의 함수는 고유한 function handle을 가짐

- Function handle 작성 방법
  - 기명법 : 생성하고자 하는 함수의 이름이 이미 존재하는 경우
  - 익명법 : 새로운 함수를 생성하고자 하는 경우

Function handle

- Function handle을 만드는 방법1: 기명법
- 함수이름 앞에 @를 붙여 function handle 생성

Funhnd = @ function_name

- 예제 :

```
>> x=1:10;
Funhnd=@mean;
Y=Funhnd(x)

Y = 5.5000
```

Function handle

- Function handle을 만드는 방법2: 익명법
- 입력변수와 수식을 나열하여 function handle 생성

  Funhnd = @(입력변수 list) 수식

- 예제 :

```
>> Asqrsum=@(x,y) x.^2+y.^2;
>> y=Asqrsum(2,3)
y = 13
```

※ 입력 변수를 제외한 모든 변수들은 상수로 저장되며, 함수가 지워지기 전까지 변하지 않는다.

Function handle

- Function handle을 만드는 방법2: 익명법

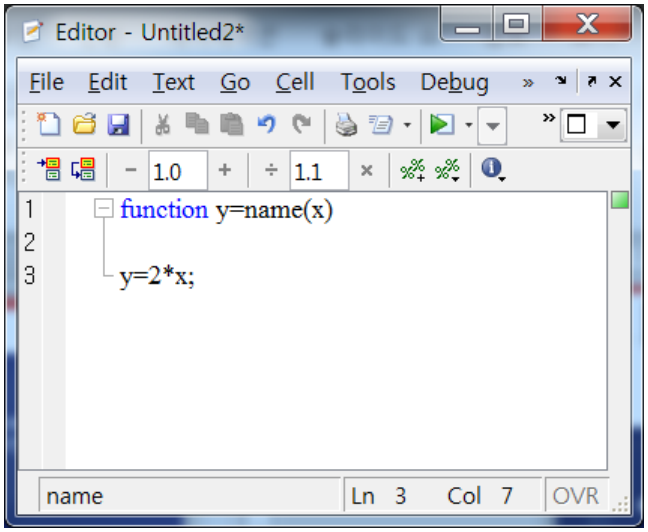
```
>> f=@(x) x.^2+c.*x+3

f =

    @(x)x.^2+c.*x+3

>> f(2)
??? Undefined function or variable 'c'.

Error in ==> @(x)x.^2+c.*x+3


>> c=3

c =

     3

>> f=@(x) x.^2+c.*x+3

f =

    @(x)x.^2+c.*x+3
```

```
>> f(2)

ans =

    13

>> c=4

c =

     4

>> f(2)

ans =

    13
```

※ 함수 **'f'**의 입력변수는 **x**이므로 함수 선언 이후에 변경된 **c**는 함수에 반영되지 않음

M-file programming

- MATLAB언어로 쓰인 파일
- Script
  - 실행파일의 형태
  - 특정 양식 없음
- Function
  - 입력→출력의 형태
  - Syntax:
  - function [output1, output2, …]=function name(input1, input2,…)



	Script M-file	Function M-file
파일이름	Anything	Function name
연산결과	모든 연산결과 기록	출력 값만 기록

M-file programming

- nargin
  - 함수의 입력변수 개수를 저장
  - nargin(@funHnd): 함수에서 정의된 입력변수의 개수를 저장
  - 입력변수가 변하는 경우 nargin은 음수
- nargout
  - 함수의 출력변수 개수를 저장
  - nargout(@funHnd): 함수에서 정의된 출력변수의 개수를 저장
  - 출력변수가 변하는 경우 nargin은 음수

※ Tip : nargin, nargout을 이용하면 입출력 변수의 개수에 따라 다른 알고리즘을 수행하는 함수 개발이 가능

M-file programming

- Global operator

global variable1 variable2 …

- 함수 공간을 가로지르기 위해 사용
- 서로 다른 함수에서 동일한 변수 사용
- 속도 저하의 가능성

```
% script: Untitled2
global mu
mu=398600.4415;

result=name(10)
```

```
function z=name(x)
global mu

y=2*x;
z=mu*y;
```

```
>> Untitled2
result =
   7.9720e+06
```

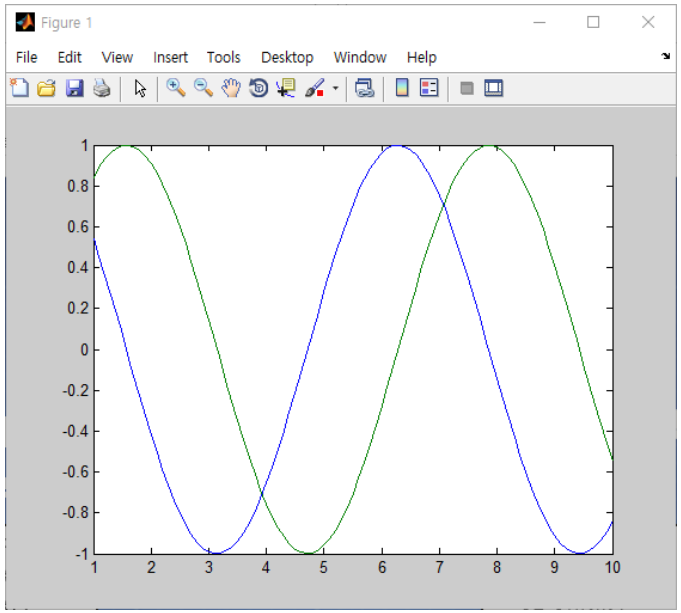
M-file programming

- 조건문
  - ➢ If/else/elseif
    - ✓ If: 논리식이 참일 경우 명령어를 수행
    - ✓ Else: 직전의 if, elseif의 논리식이 참이 아닐 때 실행
    - ✓ Elseif: if 다음에만 올 수 있으며, if의 논리식이 참이 아니면서 논리식이 참일 경우 실행
    - ✓ 모든 if 문은 반드시 end로 종료
  - ➢ For
    - ✓ index가 변하는 동안 특정 연산을 반복
    - ✓ Index가 최종 값을 가지면 최종 연산을 수행하고 종료
  - ➢ While : 논리조건이 참인 동안만 연산을 수행
  - ➢ Break: for, while문을 즉시 종료
  - ➢ Continue: for, while문에서 다음 loop로 진행

Plotting

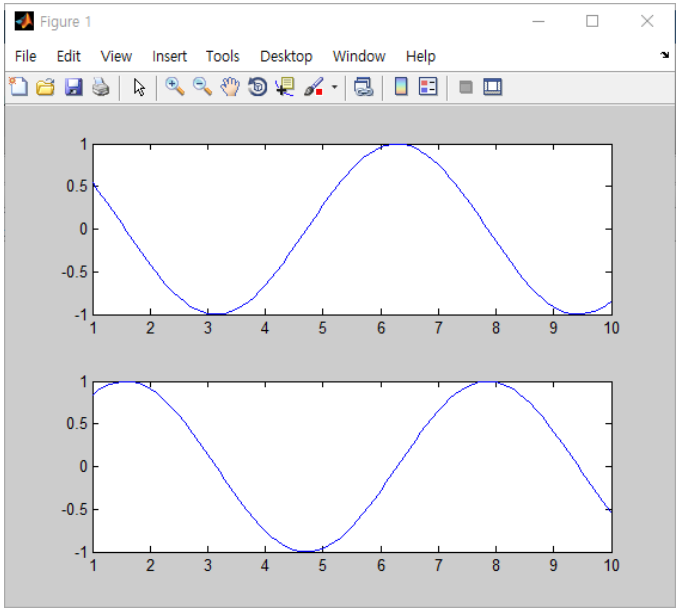
- Syntax: plot(x,y)
- Example:

```
>> x=1:0.1:10;
>> y1=cos(x);
>> y2=sin(x);
>> plot(x,y1,x,y2)
```



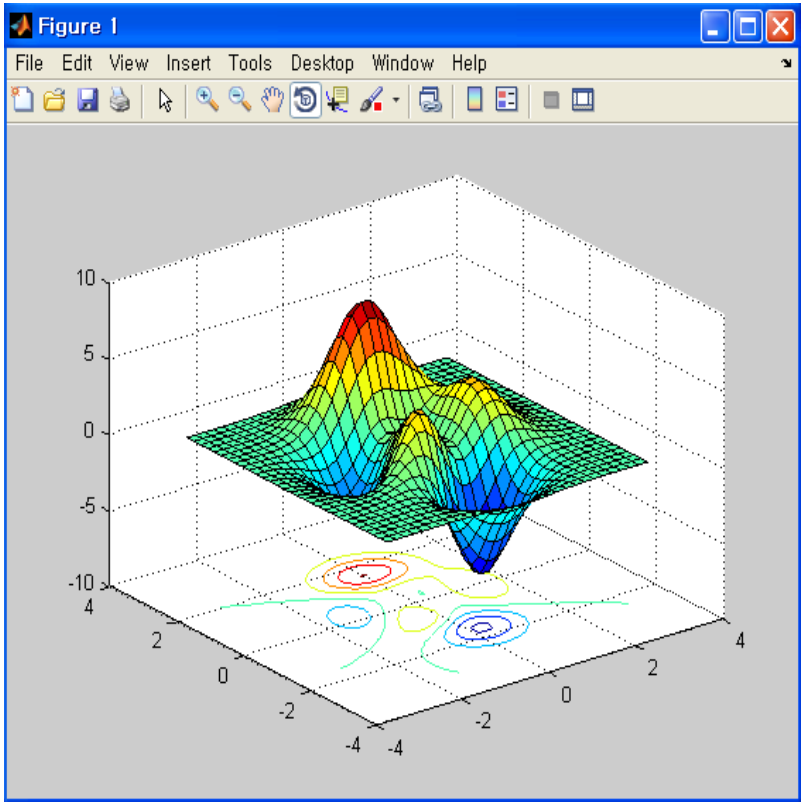
- Subplot(n,m,l)
- Example:

```
>> x=1:0.1:10;
>> y1=cos(x);
>> y2=sin(x);
>> subplot(2,1,1)
>> plot(x,y1)
>> subplot(2,1,2)
>> plot(x,y2)
```



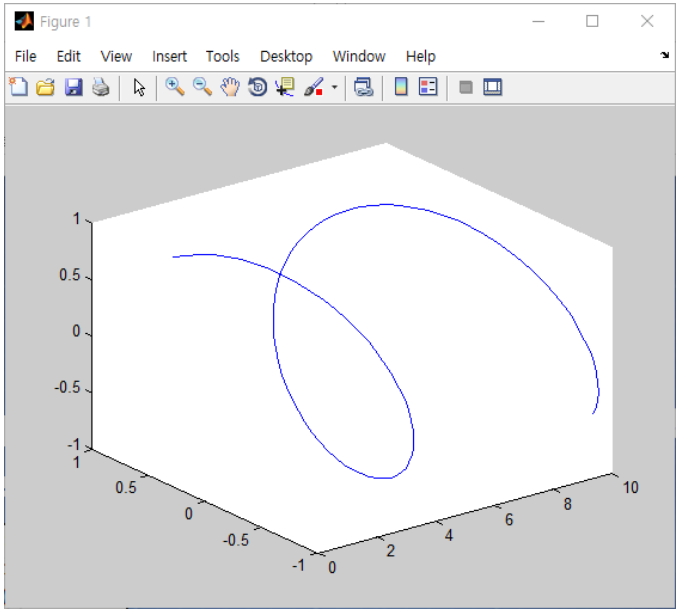
Plotting

- Syntax: Surf(x,y,z)
- Example:



- Syntax: plot3(x,y,z)
- Example:

```
>> x=1:0.1:10;
>> y1=cos(x);
>> y2=sin(x);
>> plot3(x,y1,y2)
```



Plotting options

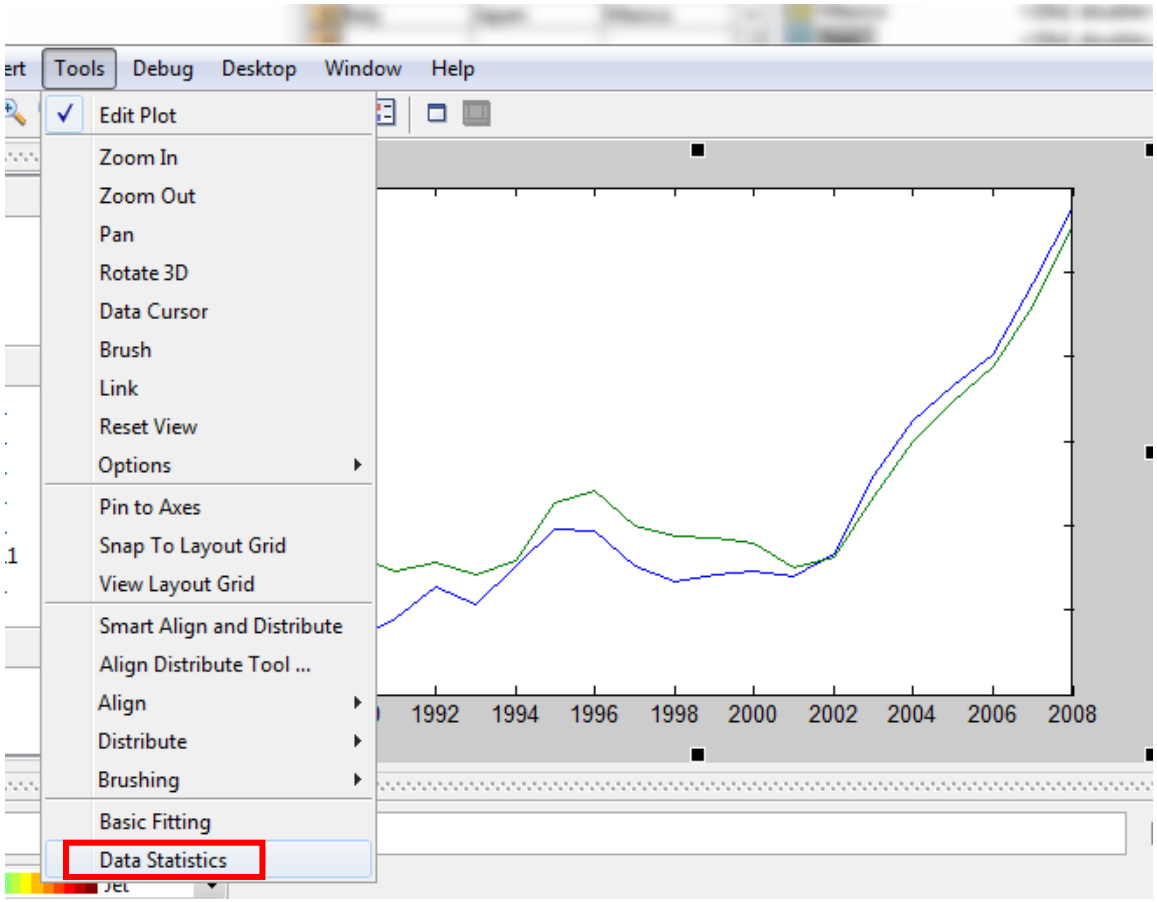
- Plotting options
  - hold on: 기존 그래프유지
  - grid on: 격자 표시
  - title: 그래프의 제목 추가
  - xlabel/ylabel/zlabel: 축 설명 추가

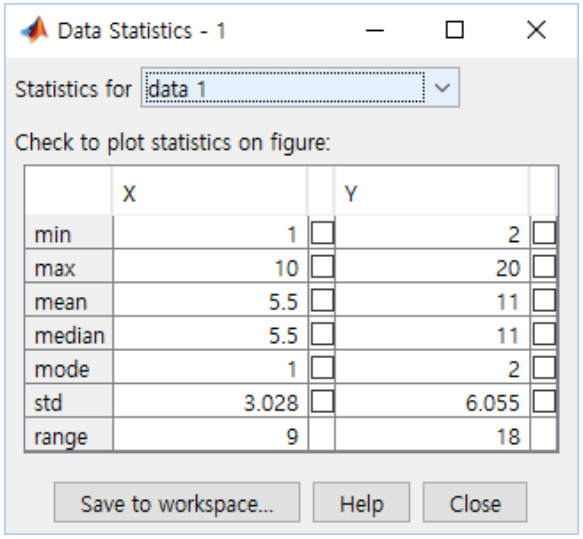
  - 그래프의 색, 모양 등 설정 가능

Type	Values	Meanings
Color	'c' 'm' 'y' 'r' 'g' 'b' 'w' 'k'	cyan magenta yellow red green blue white black
Line style	'-' '--' ':' '.-' no character	solid dashed dotted dash-dot no line
Marker type	'+' 'o' '*' 'x' 's' 'd' '^' 'v' '>' '<' 'p' 'h' no character or none	plus mark unfilled circle asterisk letter x filled square filled diamond filled upward triangle filled downward triangle filled right-pointing triangle filled left-pointing triangle filled pentagram filled hexagram no marker

Plot Tools

- Data Statistics Tool

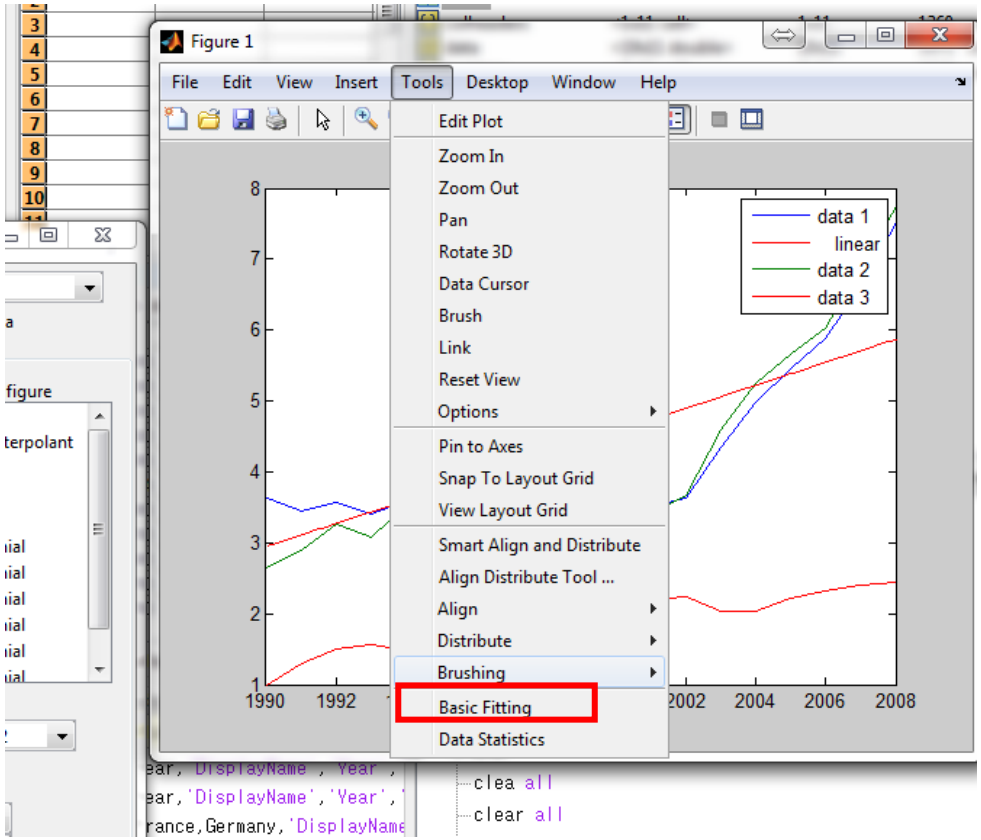


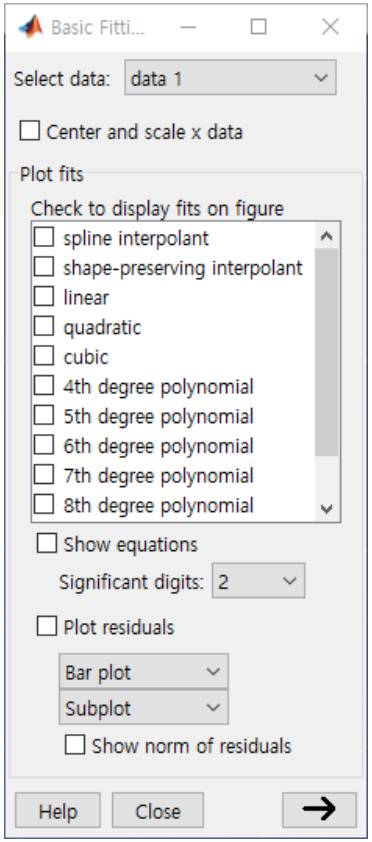


	X	Y
min	1	2
max	10	20
mean	5.5	11
median	5.5	11
mode	1	2
std	3.028	6.055
range	9	18

Plot Tools

- Basic Fitting Tool





Examples

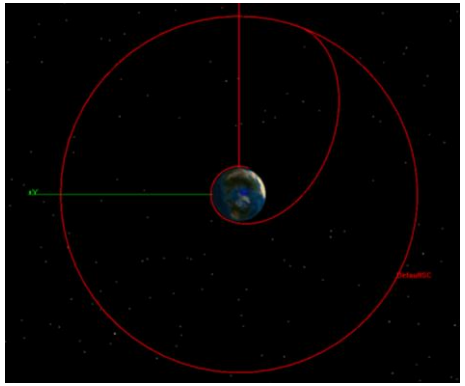
- Ex 1)
  - X가 음수이면 y=abs(x), 양수이면 y=x.^2, 0이면 y=2가 되도록 하는 m-file script
- Ex 2)
  - ✓ n!이 100보다 작도록 하는 n 가운데 가장 큰 값을 구하는 m-file script
- Ex 3)
  - 입력 값 x에 대해 x!를 계산하여 반환하는 함수 m-file function
- Ex 4)
  - X가 0~20까지 변할 때 x!을 plotting

Hohmann transfer trajectory

궤도 천이 시나리오

- 시나리오
  - ➢ 호만 천이를 활용 저궤도 위성의 정지궤도 진입

- 설정 값
  - ➢ 궤도 정보



저궤도 위성의 정지궤도 진입 예시

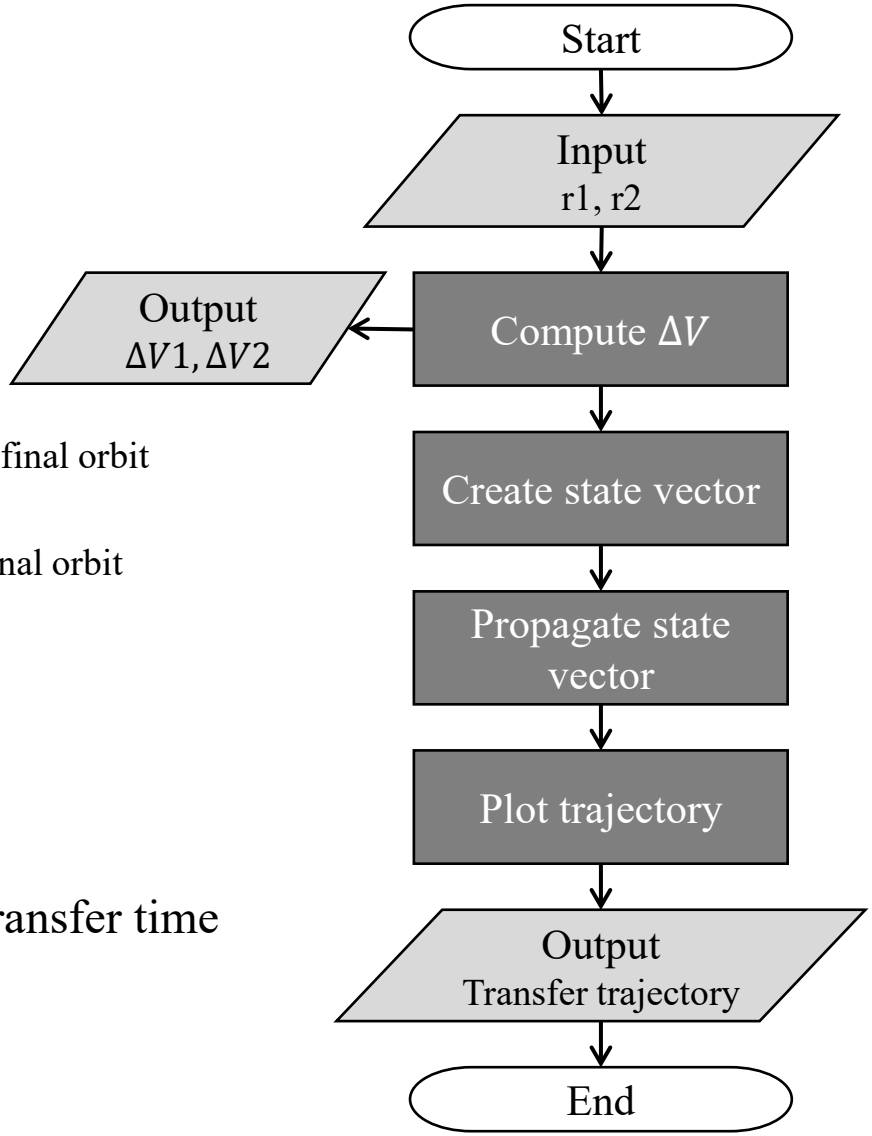
	SMA (km)	ECC	INC (deg)	RAAN (deg)	AOP (deg)	TA (deg)
초기 궤도	6700.197	0	0	0	0	20
최종 궤도	42164.197	0	0	0	0	-
천이궤도	24432	0.726	0	0	0	-

✓ 초기 기동: 2.419447 km/s
✓ 최종 기동: 1.464533 km/s

※ 참고
초기 속도: 7.7131 km/s
천이 속도 1: 10.1326 km/s
천이 속도 2: 1.6101 km/s
최종 속도 : 3.0747 km/s

Hohmann transfer trajectory

- Hohmann transfer trajectory through MATLAB
  - ➢ Input : Initial SMA, Final SMA
  - ➢ Hohmann transfer
    - ✓ Compute ΔV
      - v1, v2, $SMA_t, e_t, r_p, r_a, v_{t1}, v_{t2}$
      - $t_{trans}, \Delta V1, \Delta V2$
    - ✓ Create state vector
      - 6 orbital elements of initial, transfer, final orbit
      - 6 orbital elements -> r,v (ECI)
      - Compute period of initial, transfer, final orbit
    - ✓ Propagate state
      - **f & g**
      - 6 orbital elements
    - ✓ Plot trajectory
      - Plot 3D Hohmann transfer trajectory
  - ➢ Output : Hohmann trajectory plot, ΔV, transfer time

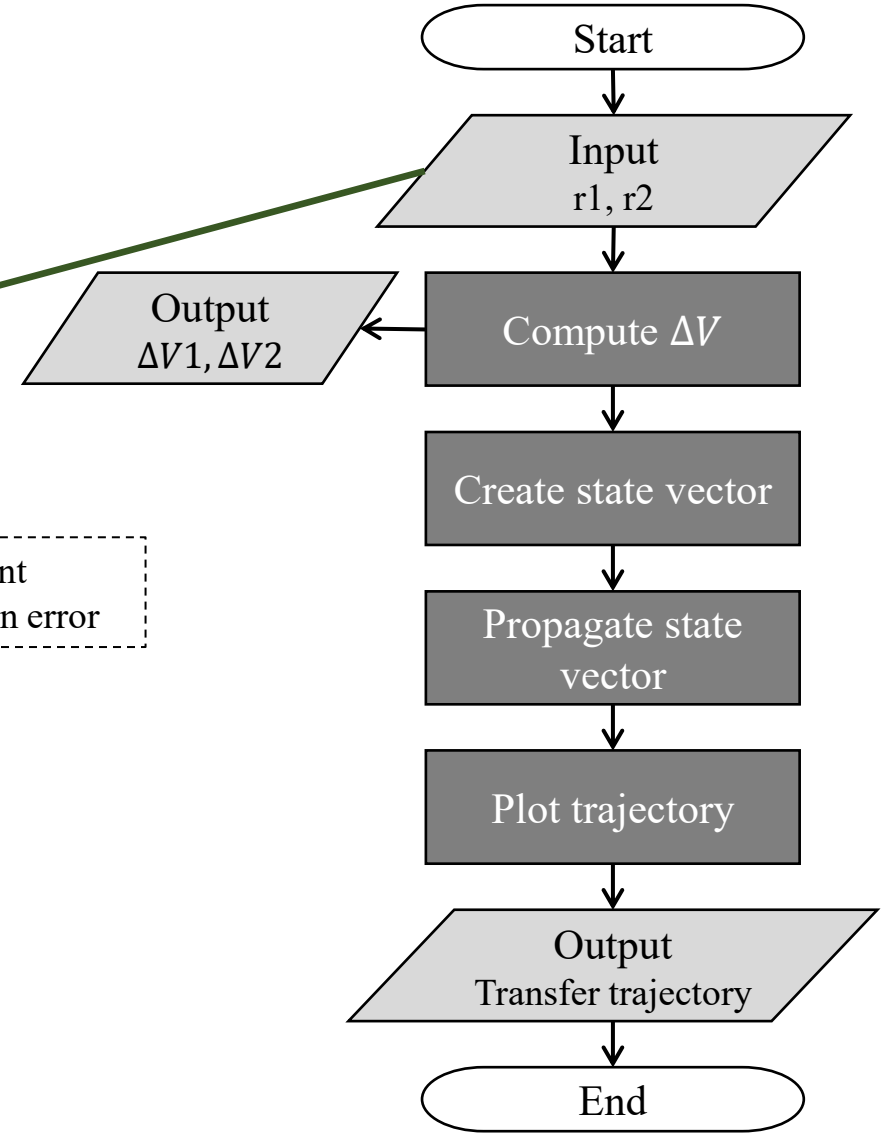


Hohmann transfer trajectory

- Hohmann transfer trajectory through MATLAB

```
clc
clear

global mu req hn1 hn2 hn3


req = 6378.1363;    % Radius of earth
r1 = 6700.1970;         %Initial radii (km)
r2 = 42164.1970;        %Final radii (km)

% compute "normalized" radii

hn1 = sqrt(2.0 * r2 / (r2 + r1));
hn2 = sqrt(r1 / r2);
hn3 = sqrt(2.0 * r1 / (r2 + r1));
```

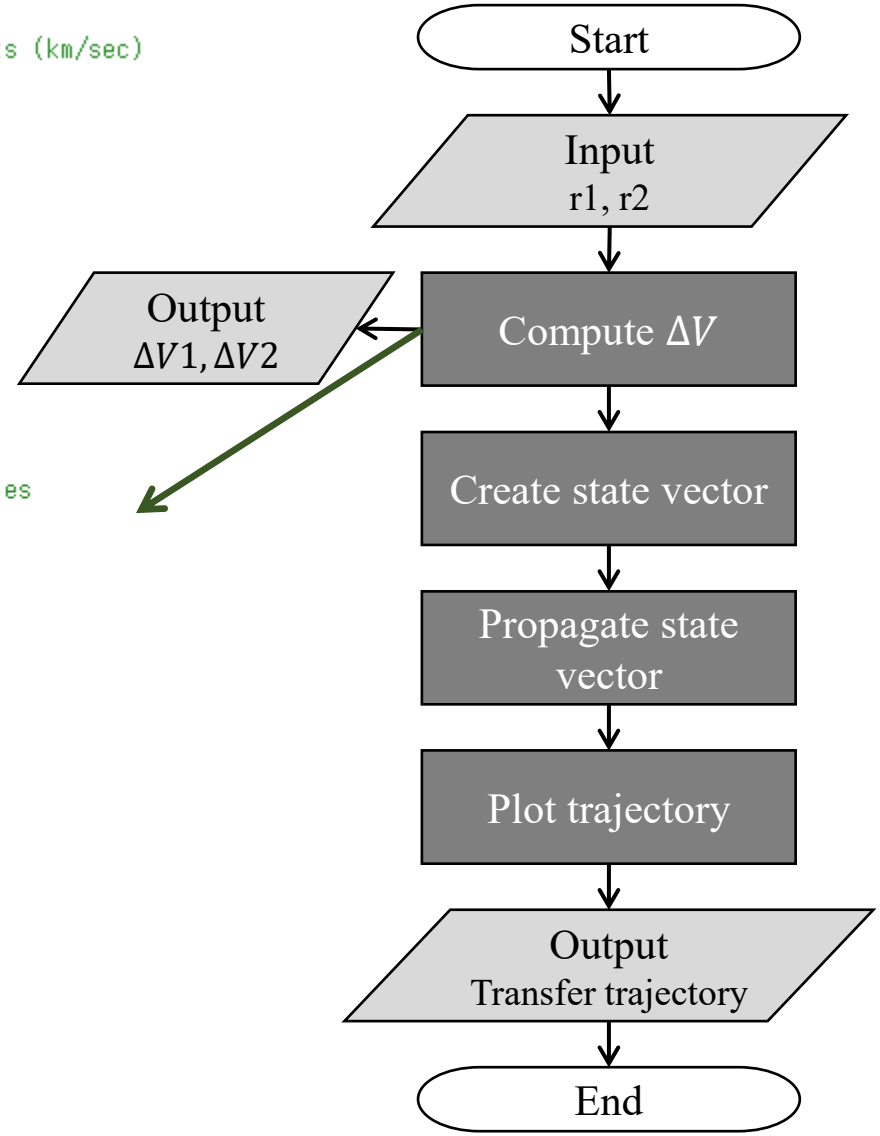
To prevent truncation error



Hohmann transfer trajectory

- Hohmann transfer trajectory through MATLAB

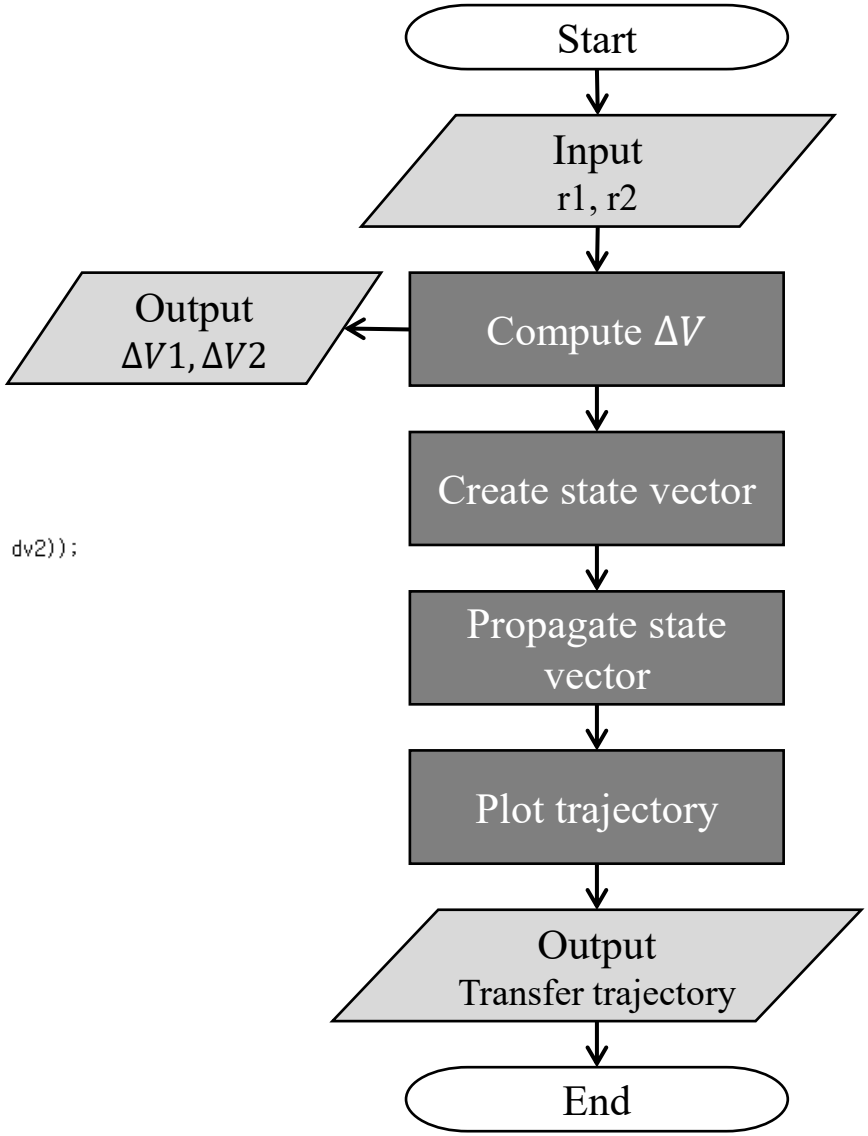
```
% compute "local circular velocity" of initial and final orbits (km/sec)
mu = 398600.436233;
v1 = sqrt(mu / r1);
v2 = sqrt(mu / r2);

% compute transfer orbit semimajor axis (kilometers)
smat = 0.5 * (r1 + r2);

% compute transfer orbit eccentricity (non-dimensional)
ecct = (max(r1, r2) - min(r1, r2)) / (r1 + r2);

% compute transfer orbit perigee and apogee radii and velocities
rp = smat * (1.0 - ecct);
ra = smat * (1.0 + ecct);
vt1 = sqrt(2.0 * mu * ra / (rp * (rp + ra)));
vt2 = sqrt(2.0 * mu * rp / (ra * (rp + ra)));

% compute transfer orbit period (seconds)

taut = 2.0 * pi * sqrt(smat^3 / mu);
tof = 0.5 * taut;

dv1 = vt1 - v1;
dv2 = v2 - vt2;
```



Hohmann transfer trajectory

- Hohmann transfer trajectory through MATLAB

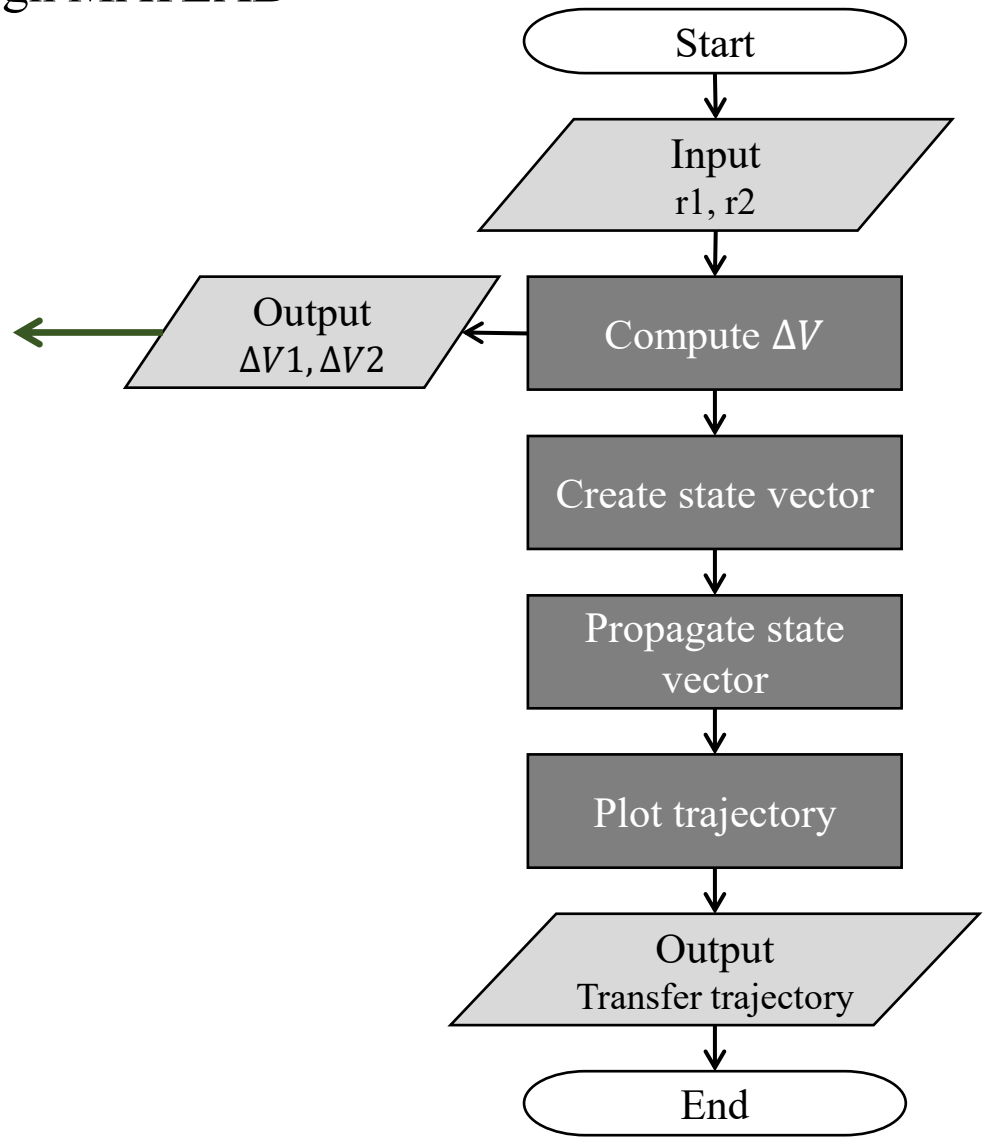
```
fprintf('\nHohmann Orbit Transfer Analysis');
fprintf('\n------------------------------\n\n');

fprintf('initial orbit radius              %10.4f kilometers \n\n', r1);

fprintf('initial orbit velocity            %10.4f meters/second \n\n\n', 1000.0 * v1);

fprintf('final orbit radius                %10.4f kilometers \n\n', r2);

fprintf('final orbit velocity              %10.4f meters/second \n', 1000.0 * v2);

fprintf('first delta-v                     %10.4f meters/second \n\n', 1000.0 * dv1);

fprintf('second delta-v                    %10.4f meters/second \n\n', 1000.0 * dv2);

fprintf('total delta-v                     %10.4f meters/second \n\n\n', 1000.0 * (dv1 + dv2));

fprintf('transfer orbit semimajor axis     %10.4f kilometers \n\n', smat);

fprintf('transfer orbit eccentricity       %10.8f \n\n', ecct);

fprintf('transfer orbit perigee velocity   %10.4f meters/second \n\n', 1000.0 * vt1);

fprintf('transfer orbit apogee velocity    %10.4f meters/second \n\n', 1000.0 * vt2);

fprintf('transfer orbit coast time         %10.4f seconds \n', tof);

fprintf('                                  %10.4f minutes \n', tof / 60.0);

fprintf('                                  %10.4f hours \n\n', tof / 3600.0);
```



Hohmann transfer trajectory

- Hohmann transfer trajectory through MATLAB

```
Hohmann Orbit Transfer Analysis
-------------------------------

initial orbit radius              6700.1970 kilometers

initial orbit velocity            7713.0314 meters/second


final orbit radius               42164.1970 kilometers

final orbit velocity              3074.6591 meters/second
first delta-v                     2419.4486 meters/second

second delta-v                    1464.5345 meters/second

total delta-v                     3883.9831 meters/second


transfer orbit semimajor axis    24432.1970 kilometers

transfer orbit eccentricity      0.72576363

transfer orbit perigee velocity  10132.4800 meters/second

transfer orbit apogee velocity    1610.1246 meters/second

transfer orbit coast time        19003.1138 seconds
                                   316.7186 minutes
                                     5.2786 hours
```



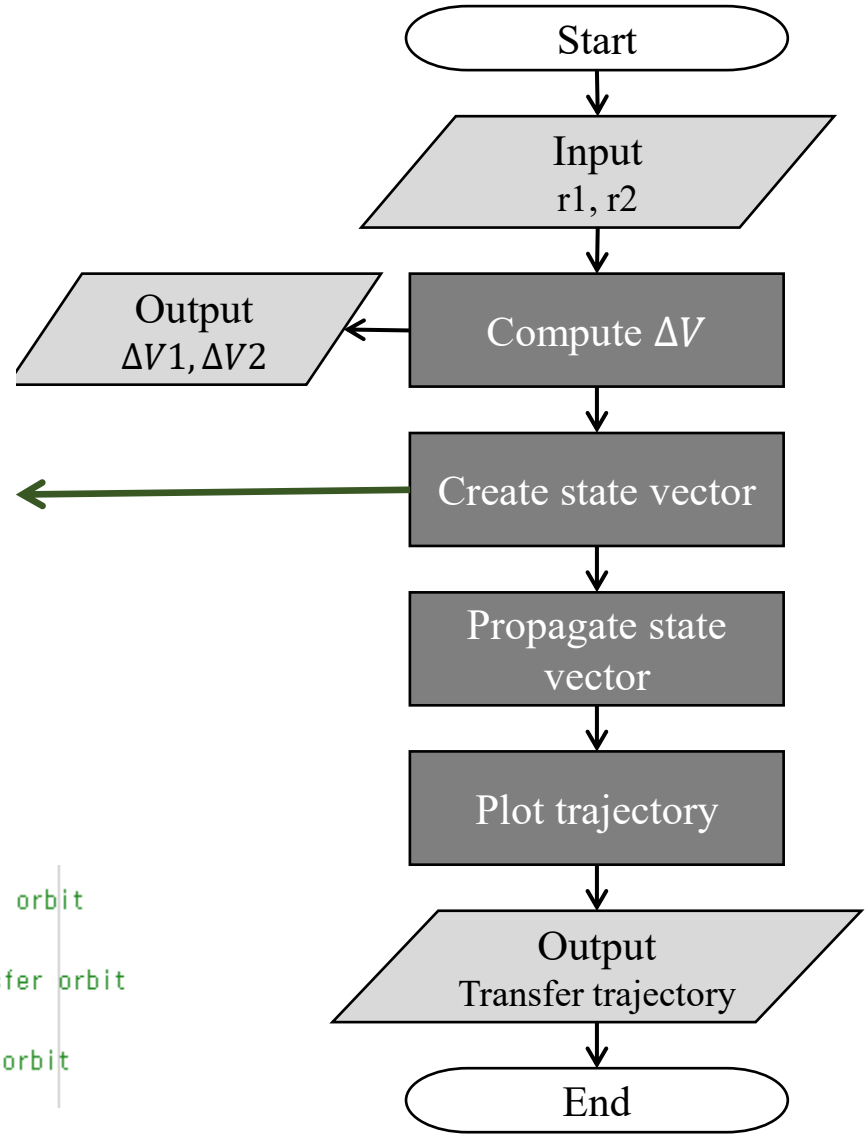
Hohmann transfer trajectory

- Hohmann transfer trajectory through MATLAB

```
% Initial orbit
oevi(1) = r1;
oevi(2) = 0.0;
oevi(3) = 0.0;
oevi(4) = 0.0;
oevi(5) = 0.0;
oevi(6) = 0.0;
[ri, vi] = orb2eci(mu, oevi);

%Transfer orbit_first burn
oevti(1) = smat;
oevti(2) = ecct;
oevti(3) = 0.0;
oevti(4) = 0.0;
oevti(5) = 0.0;
oevti(6) = 0.0;
[rti, vti] = orb2eci(mu, oevti);
```

```
%Transfer orbit_second burn
oevtf(1) = smat;
oevtf(2) = ecct;
oevtf(3) = 0.0;
oevtf(4) = 0.0;
oevtf(5) = 0.0;
oevtf(6) = 180.0;
[rtf, vtf] = orb2eci(mu, oevtf);

%Final orbit
oevf(1) = r2;
oevf(2) = 0.0;
oevf(3) = 0.0;
oevf(4) = 0.0;
oevf(5) = 0.0;
oevf(6) = 180.0;
[rf, vf] = orb2eci(mu, oevf);
```

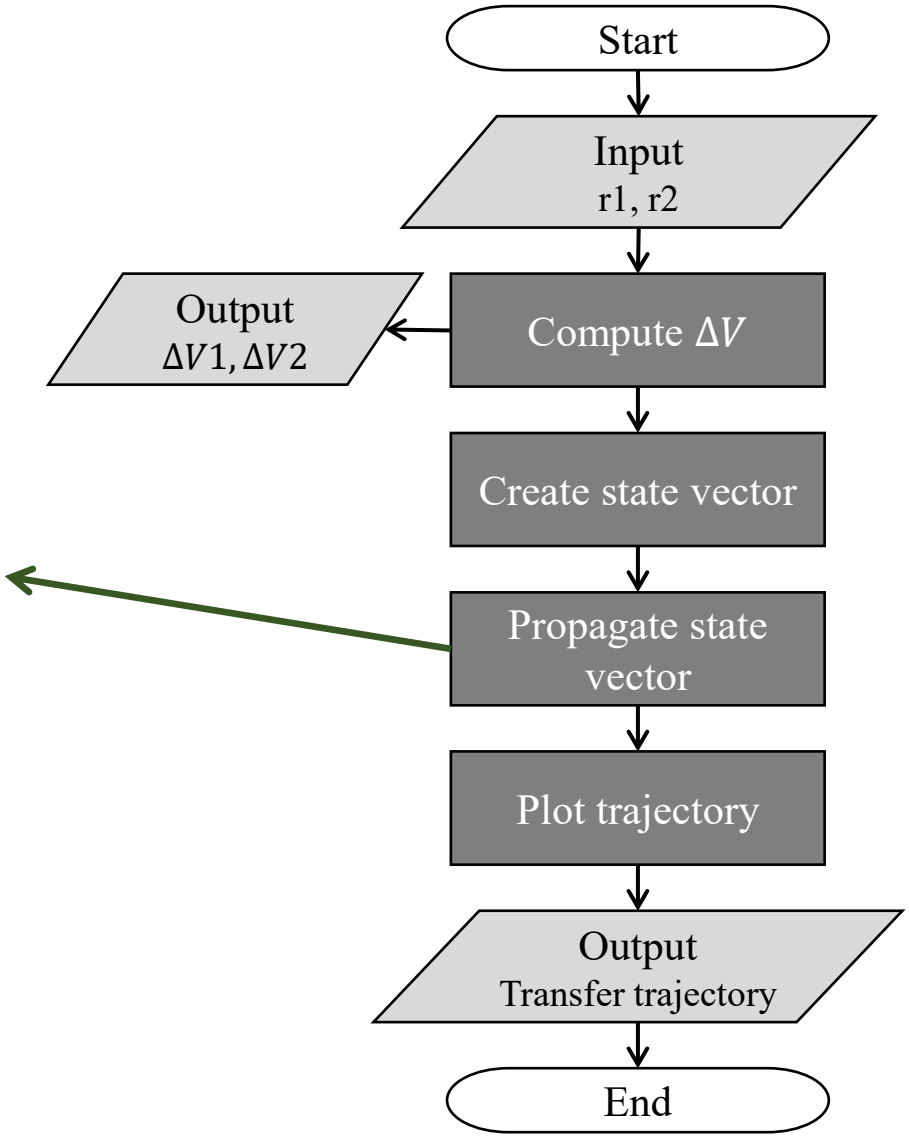
```
period1 = 2.0 * pi * oevi(1) * sqrt(oevi(1) / mu);  % Period of initial orbit

period2 = 2.0 * pi * oevti(1) * sqrt(oevti(1) / mu);  % Period of transfer orbit

period3 = 2.0 * pi * oevf(1) * sqrt(oevf(1) / mu);   % Period of final orbit
```



Hohmann transfer trajectory

- Hohmann transfer trajectory through MATLAB

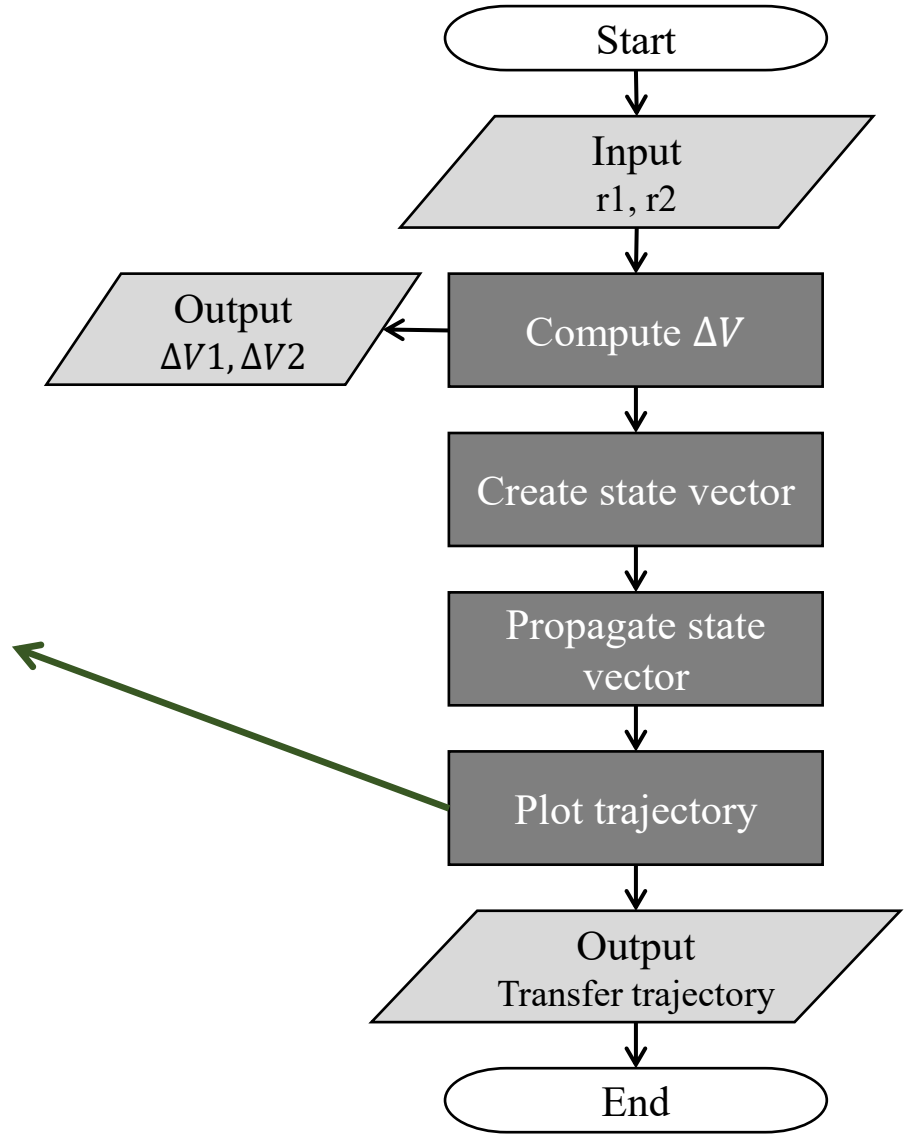
```
% Divide period for simulation
deltat1 = period1 / 300;
simtime1 = -deltat1;
deltat2 = 0.5 * period2 / 300;
simtime2 = -deltat2;
deltat3 = period3 / 300;
simtime3 = -deltat3;

for i = 1:1:301
    simtime1 = simtime1 + deltat1;
    simtime2 = simtime2 + deltat2;
    simtime3 = simtime3 + deltat3;

    % compute initial orbit "normalized" position vector
    [rwrk, vwrk] = twobody2 (mu, simtime1, ri, vi);
    rp1_x(i) = rwrk(1) / req;
    rp1_y(i) = rwrk(2) / req;
    rp1_z(i) = rwrk(3) / req;

    % compute transfer orbit position vector
    [rwrk, vwrk] = twobody2 (mu, simtime2, rti, vti);
    rp2_x(i) = rwrk(1) / req;
    rp2_y(i) = rwrk(2) / req;
    rp2_z(i) = rwrk(3) / req;

    % compute final orbit posi;tion vector
    [rwrk, vwrk] = twobody2 (mu, simtime3, rf, vf);
    rp3_x(i) = rwrk(1) / req;
    rp3_y(i) = rwrk(2) / req;
    rp3_z(i) = rwrk(3) / req;
end
```



Hohmann transfer trajectory

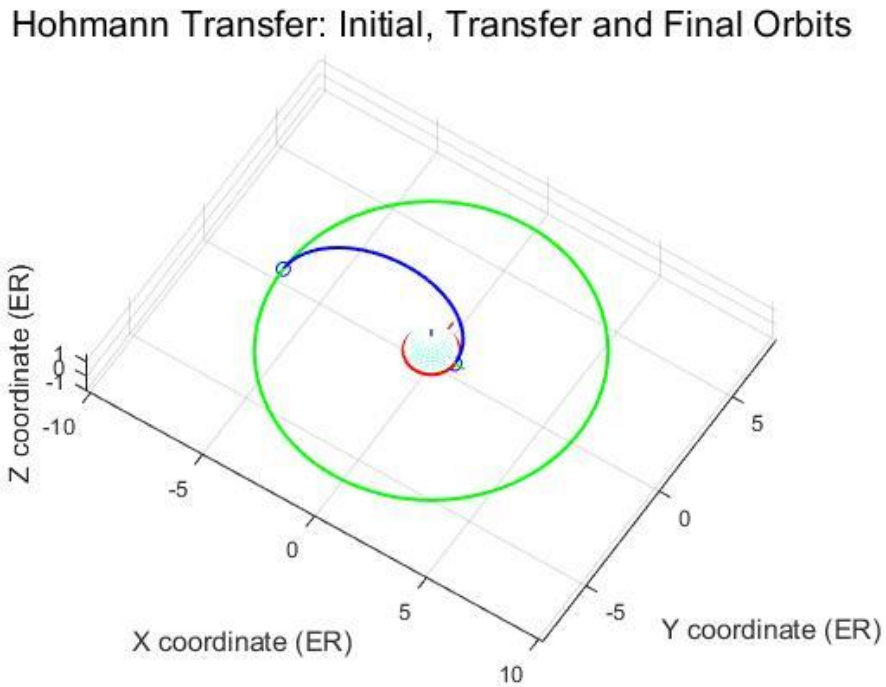
- Hohmann transfer trajectory through MATLAB

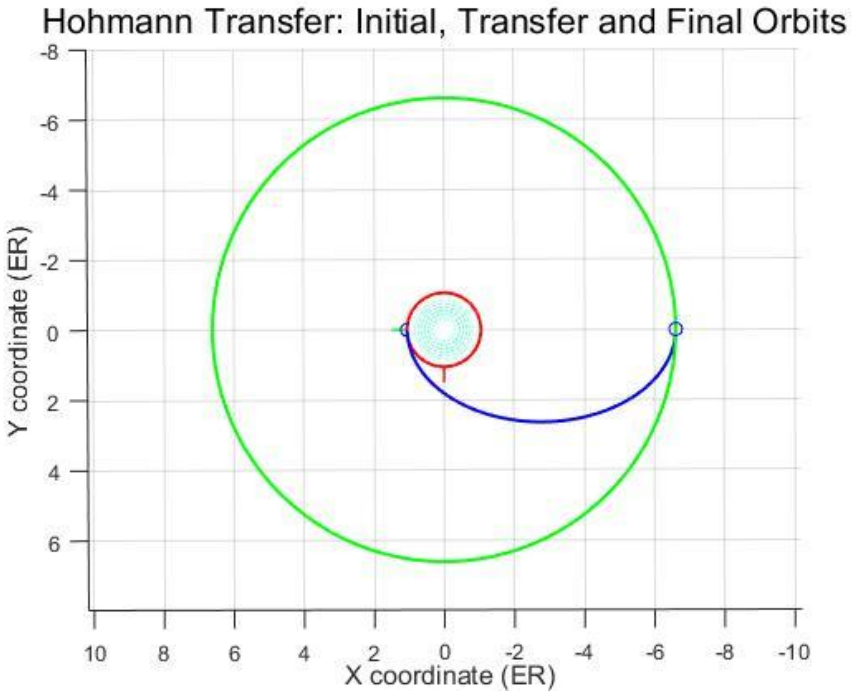
```
figure(1);
% create axes vectors
xaxisx = [1 1.5];
xaxisy = [0 0];
xaxisz = [0 0];

yaxisx = [0 0];
yaxisy = [1 1.5];
yaxisz = [0 0];

zaxisx = [0 0];
zaxisy = [0 0];
zaxisz = [1 1.5];

figure (1);
hold on;
grid on;
% plot earth
[x y z] = sphere(24);
h = surf(x, y, z);
colormap([127/255 1 222/255]);
set (h, 'edgecolor', [1 1 1]);

% plot coordinate system axes
plot3(xaxisx, xaxisy, xaxisz, '-g', 'LineWidth', 1);
plot3(yaxisx, yaxisy, yaxisz, '-r', 'LineWidth', 1);
plot3(zaxisx, zaxisy, zaxisz, '-b', 'LineWidth', 1);
% plot initial orbit
plot3(rp1_x, rp1_y, rp1_z, '-r', 'LineWidth', 1.5);
plot3(rp1_x(1), rp1_y(1), rp1_z(1), 'ob');
% plot transfer orbit
plot3(rp2_x, rp2_y, rp2_z, '-b', 'LineWidth', 1.5);
plot3(rp2_x(end), rp2_y(end), rp2_z(end), 'ob');
% plot final orbit
plot3(rp3_x, rp3_y, rp3_z, '-g', 'LineWidth', 1.5);
xlabel('X coordinate (ER)', 'FontSize', 12);
ylabel('Y coordinate (ER)', 'FontSize', 12);
zlabel('Z coordinate (ER)', 'FontSize', 12);
title('Hohmann Transfer: Initial, Transfer and Final Orbits', 'FontSize', 16);

axis equal;
view(50, 20);
rotate3d on;
print -depsc -tiff -r300 hohmann1.eps
```



Hohmann transfer trajectory

- Hohmann transfer trajectory through MATLAB





Thank you for listening